There are many options to build a baseline model. Our personal favorite way to begin the experimentation phase of a machine learning task is to use AutoML in the Databricks Platform. AutoML experiments are easy to set up and execute, automatically provide a ton of information on a model's performance, and autogenerate data products like the data exploration notebook. The complete set of code and modeling artifacts is the "glass-box" concept in action, enabling you to recreate, alter, and fully own the results. We used AutoML in *Chapter 4* to explore the Favorita Sales data via the data profile provided in the data exploration notebook. You may have noticed the model run information when generating the exploration notebook, which is what we will focus on now.

AutoML rapidly explores many model-algorithm/hyperparameter permutations to find the best baseline model for your data, along with tracking evaluation metrics and actual code for each run. Once you have created a baseline model, you can evaluate future models against its performance metrics, such as accuracy, precision, recall, confusion matrices, ROC curves, and AUC. As great as it is, we also caution that AutoML is not a magic bullet. It removes some of the overhead of coding multiple algorithms for experimentation, but you are still in charge of where to go next. Luckily, AutoML automatically tracks these model artifacts in Databricks.

Tracking the many features, models, hyperparameters, and evaluation metrics is a real headache, so using MLflow to track everything natively is a lifesaver. In the *Applying our learning* section of this chapter, we use AutoML to build traditional regression baseline models for forecasting. Next, let's explore MLflow Tracking, so we know how to apply it later in *Applying our learning*.

Tracking experiments with MLflow

MLflow is an open-source platform developed by Databricks for managing data science projects. MLflow supports data science practitioners through the entire machine learning life cycle, from the experimentation phase to packaging code to model deployment in production. Because the platform is open-source, it benefits from a robust open-source development community and continues to expand to support new features and models, including in the generative AI space. Within the DI Platform, we use a managed version of MLflow that natively integrates with the rest of the Databricks platform. We will cover the deployment-related components in *Chapter 7*. For now, let's focus on tracking.

Before we had MLflow Tracking, managing machine learning experiments required much work outside of the actual experimentation process. Tracking handles the overhead of capturing the configurations of environment, features, algorithms, and hyperparameters, plus their corresponding evaluation metrics during testing. Whenever you execute an experiment with MLflow in a notebook, every execution of that experiment is tracked with MLflow Tracking. Consequently, you can use the MLflow UI to compare experiments. We discussed this in the "Modeling in Databricks" section of *Chapter 2*. For convenience, Databricks notebooks have integrations that make it easy to manage and compare experiment results programmatically via MLflow's lightweight APIs or the Databricks notebook UI. For example, *Figure 6.2* shows how to view experiment runs while still in a notebook, by clicking the small Erlenmeyer Flask icon on the screen's far right.